BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

LESSON 183 · Recursion: write and debug algorithms

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. In SumTo(4), which call returns first and which returns last?

Q2. Why must a recursive function contain a stopping condition?

Q3. A loop repeatedly halves n starting at 40 while n > 1 using integer division. List the next three n values.

Q4. A function PowerTwo(n) returns 1 when n=0, otherwise 2*PowerTwo(n-1). What does PowerTwo(3) return?

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

termination — Reaching a condition where the algorithm stops.

recursive step — Processing part of a problem before calling the function on a simpler input.

precondition — An assumption or input rule required for an algorithm.

stack overflow — A failure that can occur when too many active calls consume stack space.

integer division — Division discarding any fractional part of the result.

New learning

Find the smallest problem — A positive single-digit number needs one digit. So n < 10 is our base case.

Make progress — For n ≥ 10, integer division by 10 makes the number shorter: 4827 → 482 → 48 → 4.

Count on the way back — 4 returns 1. Then 48 returns 2; 482 returns 3; 4827 returns 4.

Worked example

Model CountDigits(582). Why is integer division essential?

1. Calls — 582 → 58 → 5. Each integer division by 10 discards the last digit.

2. Base case — 5 is a single digit, so CountDigits(5) returns 1.

3. Returns — CountDigits(58)=2; CountDigits(582)=3. The original number has 3 digits.

We Do · Practise with guidance

Trace CountDigits(9031). Predict each call and returned value.

9031 → 903 → 90 → 9

Predict the next step before revealing the explanation.

Calls

9031, 903, 90, 9.

Base case

9 is a single digit so returns 1.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

RHS COMPUTING / OCR H446 / 2.2.1 LESSON 183 · DRAFT

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Returns

9 returns 1; 90 returns 2; 903 returns 3; 9031 returns 4.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Trace CountDigits(67): write the call sequence and final value.

Question 2

A pupil writes return CountDigits(n) + 1; for the recursive step. Correct the line and explain the error.

Question 3

Extend CountDigits for n = 0 and negative integers without overflowing on the minimum C# int value.

Exit Ticket · Answer before checking

An algorithm is recursive whenever it uses a loop.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. SumTo(1) returns first; SumTo(4) returns last after its nested calls unwind.

Q2. Without a reachable base case, calls continue and the call stack grows until failure.

Q3. 20, 10, 5.

Q4. PowerTwo(0)=1, PowerTwo(1)=2, PowerTwo(2)=4 and PowerTwo(3)=8.

You Do pages · Independent questions

Q1. Calls 67 → 6. Base case returns 1, then the outer call returns 2.

Q2. Correct: return 1 + CountDigits(n / 10);. The original line does not reduce n, so it never reaches the base case for n >= 10.

Q3. An example robust implementation: int CountDigits(int n) { long value = Math.Abs((long)n); if (value < 10) return 1; return 1 + CountDigits((int)(value / 10)); } For the most-negative int, the cast is safe here because dividing by 10 reduces the magnitude first.

Exit Ticket · Model answer

False. Recursion requires a function to call itself; iteration uses a loop.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: